The expectations use match_array because you haven't specified an ordering, and match_array will work no matter what order the arrays are in. The admin test checks against Project.all to protect against other projects being added by fixtures or tests that might run earlier in the file.

The last test is the direct result of realizing that the code as I left it following the test before might duplicate an entry if a project were both public and had the user as a member—though in fact, Rails 5 makes it easy to handle this.

Here's the "fast to green" passing code:

security/04/app/models/user.rb

```
def visible_projects
  return Project.all if admin?
  Project.where(id: project_ids).or(Project.where(public: true))
end
```

This method returns a scope rather than an array, so it can be chained with other ActiveRecord code.

First off, in your refactor, move the Project logic to its own class method:

security/05/app/models/project.rb

```
def self.all_public
  where(public: true)
end
```

This makes the visible_projects method look like this:

security/05/app/models/user.rb

```
def visible_projects
  return Project.all if admin?
  Project.where(id: project_ids).or(Project.all_public)
end
```

The User code has a larger problem. It now has two methods, can_view? and visible_projects, that duplicate the logic of whether a user can view a project. One possible solution is to rewrite can_view? in terms of visible_projects:

security/05/app/models/user.rb

```
def can_view?(project)
  project.in?(visible_projects)
end
```

If you do that, you have some test failures—one test that didn't update a project's status to the database now needs to. Actually, you have a test refactoring here—two sets of tests covering the same logic. Let's combine them to make it even more clear that the two methods are in parallel: